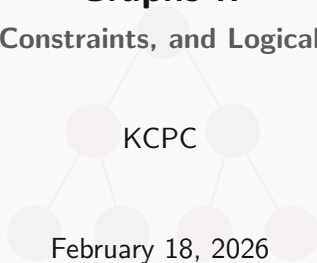


Graphs II

Distance, Constraints, and Logical Structure



Overview

- 1 Foundations - A Quick Graph Traversal Recap
- 2 Dijkstra - An Exploration
- 3 0-1 BFS - An Exploration
- 4 Choosing the Right Algorithm (Quick Rules)
- 5 Problem Sheet
- 6 Final Remarks



Attendance



Our Socials



Instagram



Discord



WhatsApp

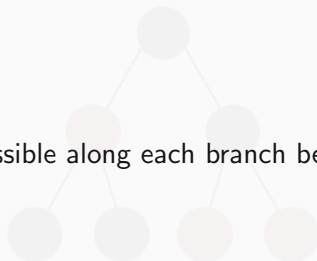
Foundations - A Quick Graph Traversal Recap



Depth-First Search

Purpose

Explore as far as possible along each branch before backtracking.



Depth-First Search

Algorithm (Iterative)

1. Initialize

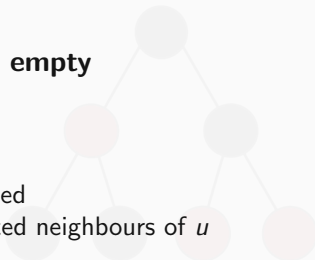
- Create empty stack
- Mark all nodes unvisited
- Push start node



Depth-First Search

2. While stack not empty

- Pop node u
- If u unvisited:
 - Mark u visited
 - Push unvisited neighbours of u

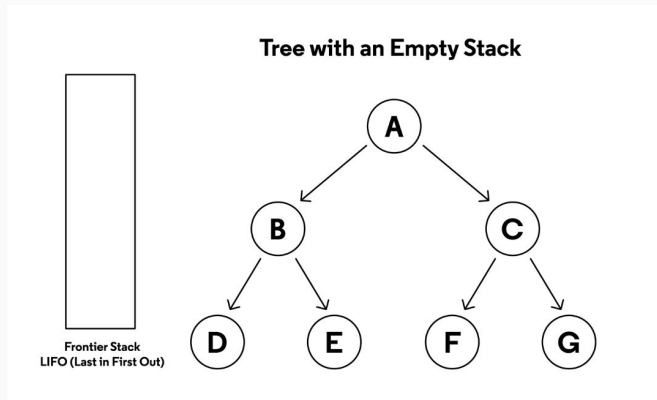


Depth-First Search

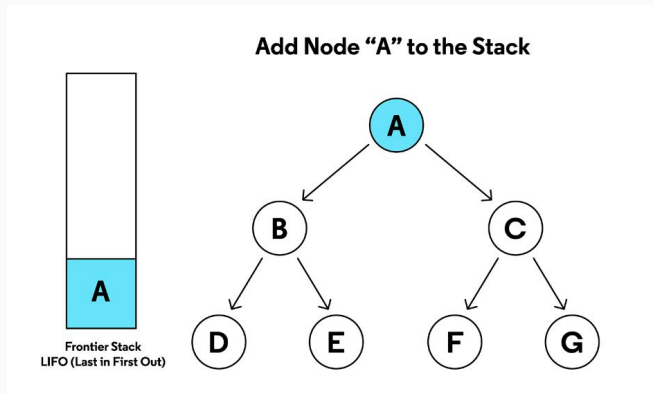
3. Stop when stack empty



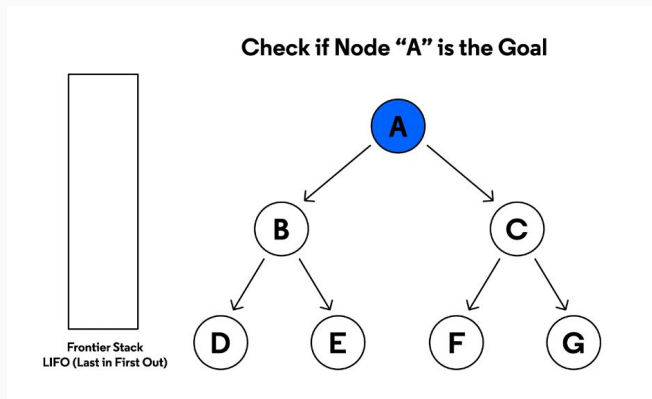
Group walkthrough example



Group walkthrough example

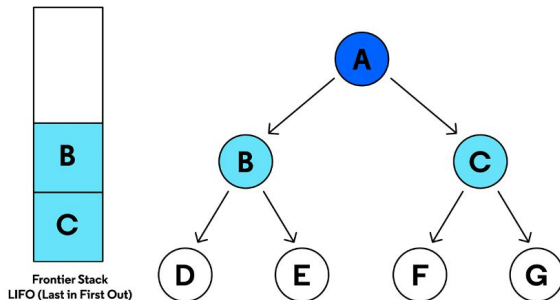


Group walkthrough example



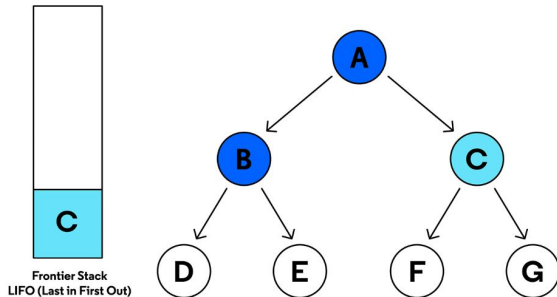
Group walkthrough example

Add Children of Node "A" to the Stack



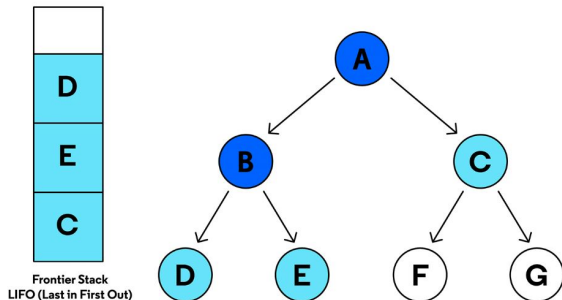
Group walkthrough example

Check if Node "B" is the Goal



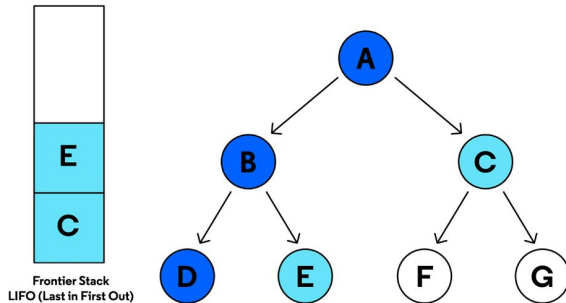
Group walkthrough example

Add Children of Node "B" to the Stack



Group walkthrough example

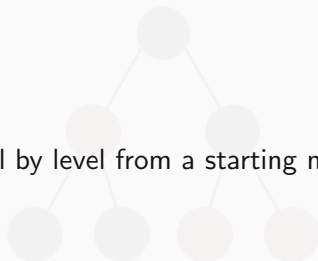
Check if Node "D" is the Goal



Breadth-First Search (BFS)

Purpose

Explore a graph level by level from a starting node.

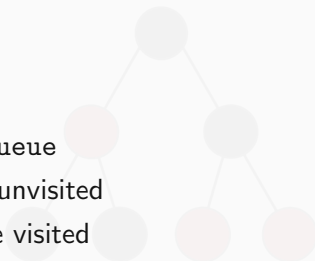


Breadth-First Search (BFS)

Algorithm

1. Initialize

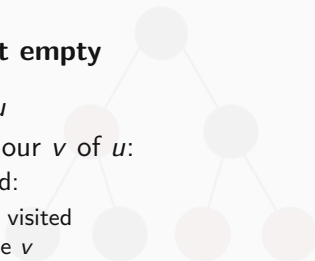
- Create empty queue
- Mark all nodes unvisited
- Mark start node visited
- Enqueue start node



Breadth-First Search (BFS)

2. While queue not empty

- Dequeue node u
- For each neighbour v of u :
 - If v unvisited:
 - Mark v visited
 - Enqueue v

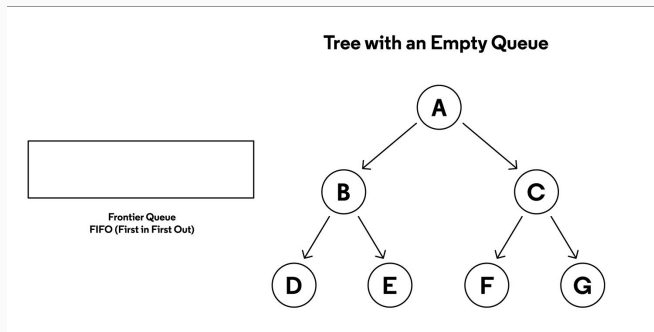


Breadth-First Search (BFS)

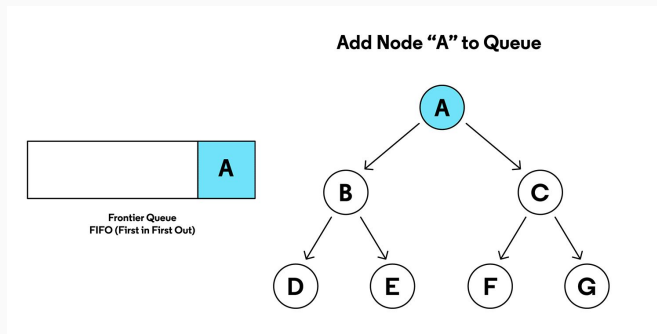
3. Stop when queue empty



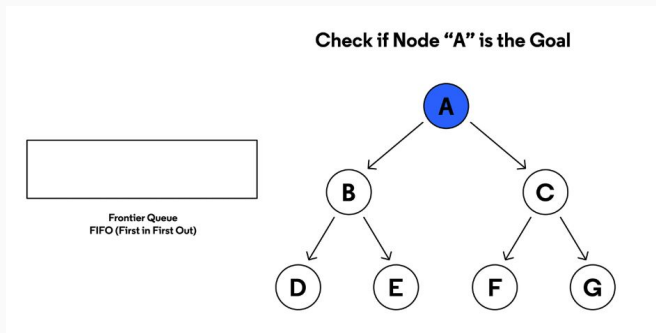
Group walkthrough example



Group walkthrough example



Group walkthrough example

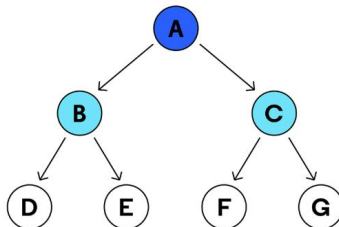


Group walkthrough example

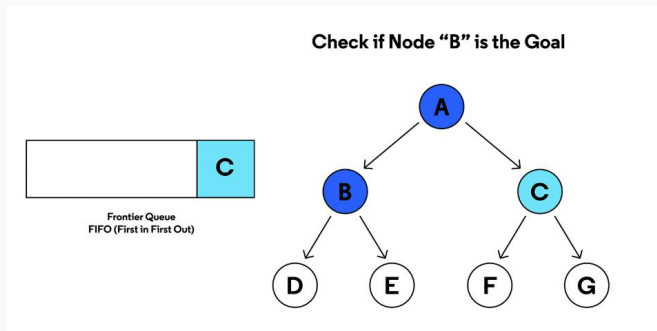
Add Children of Node "A" to the Queue



Frontier Queue
FIFO (First in First Out)



Group walkthrough example

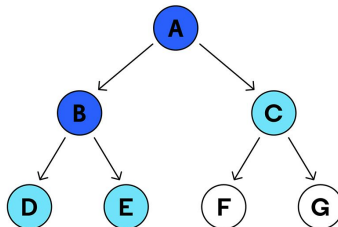


Group walkthrough example

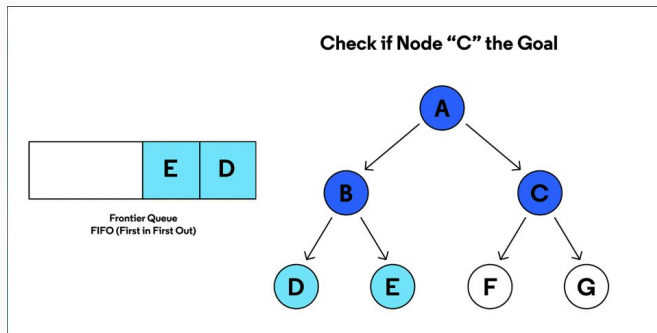
Add Children of Node "B" to Queue



Frontier Queue
FIFO (First in First Out)



Group walkthrough example

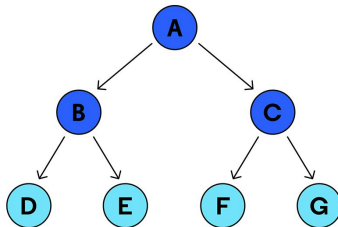


Group walkthrough example

Add Children of Node "C" to the Queue



Frontier Queue
FIFO (First in First Out)



Why BFS Is Not Enough

Edge Case: Weighted Graph

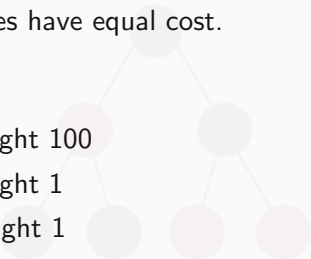
BFS assumes all edges have equal cost.

Consider:

→ $A \rightarrow B$ with weight 100

→ $A \rightarrow C$ with weight 1

→ $C \rightarrow B$ with weight 1



What BFS does:

→ Finds path $A \rightarrow B$ (1 edge)

Why BFS Is Not Enough

Actual shortest path:

→ $A \rightarrow C \rightarrow B$ with total cost 2

Key Limitation

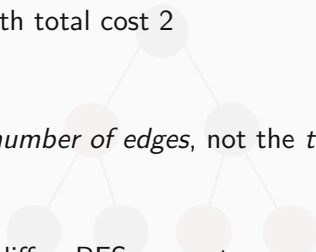
BFS minimises the *number of edges*, not the *total path weight*.

Implication

When edge weights differ, BFS may return a suboptimal path.

Takeaway Point

Do not use BFS on graphs with weights on them!



Priority Queues

Motivation

When edge weights differ, we must expand the *closest* node first.

Key Idea

Instead of processing nodes in FIFO order (like BFS), we process nodes in order of *smallest current distance*.

Data Structure

A min-heap (priority queue) allows us to:

- Always extract the node with smallest distance
- Efficiently update distances

Dijkstra - An Exploration



Why Would We Ever Use This?

Problem

BFS only guarantees shortest path when all edges have equal weight.

Issue

When edge weights differ:

- The path with fewer edges may not have minimum total cost
- We must prioritise smaller cumulative distance

Solution

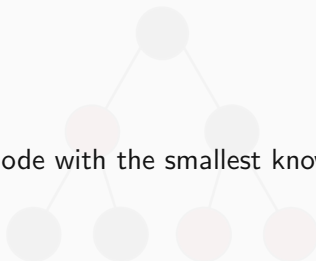
Use Dijkstra's Algorithm:

- Always expand the node with smallest known distance
- Guarantees shortest paths for non-negative weights

Algorithmic Theory

Core Idea

Always expand the node with the smallest known distance.

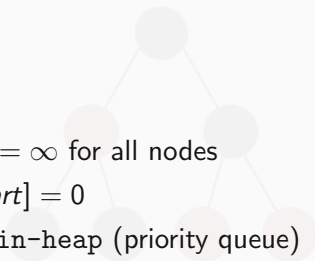


Algorithmic Theory

Algorithm

1. Initialize

- Set $distance[v] = \infty$ for all nodes
- Set $distance[start] = 0$
- Create empty min-heap (priority queue)
- Insert $(0, start)$ into queue



Algorithmic Theory

2. While queue not empty

- Extract node u with smallest distance
- If extracted distance is outdated, skip
- For each neighbour v with edge weight w :
 - If $distance[u] + w < distance[v]$:
 - Update $distance[v]$
 - Insert $(distance[v], v)$ into queue

Algorithmic Theory

3. Stop when queue empty



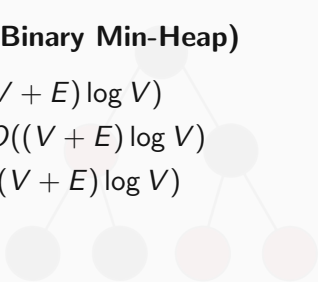
Algorithmic Theory

Time Complexity (Binary Min-Heap)

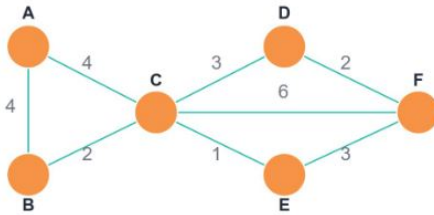
- Best case: $O((V + E) \log V)$
- Average case: $O((V + E) \log V)$
- Worst case: $O((V + E) \log V)$

Space Complexity

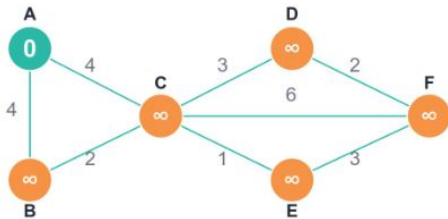
- $O(V + E)$



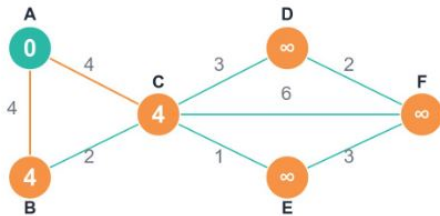
Group walkthrough example



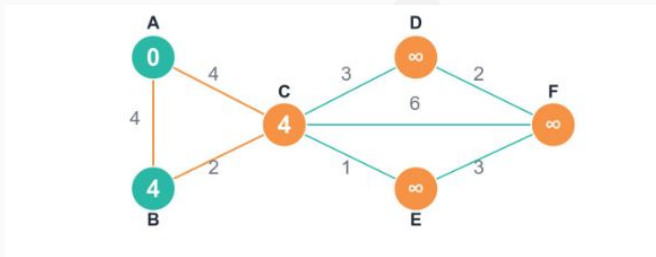
Group walkthrough example



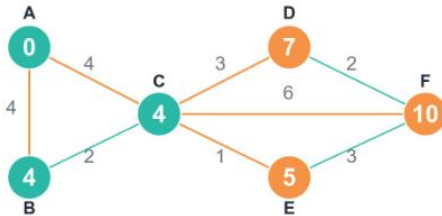
Group walkthrough example



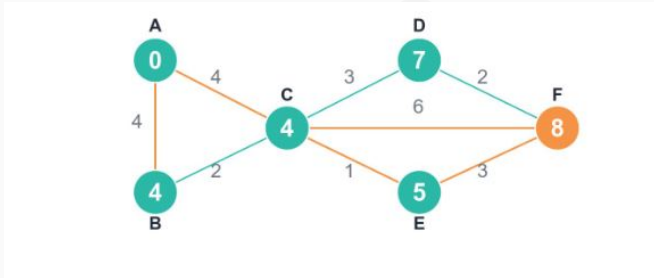
Group walkthrough example



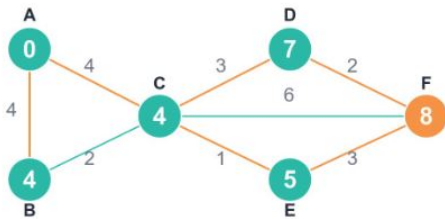
Group walkthrough example



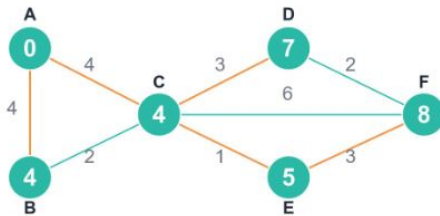
Group walkthrough example



Group walkthrough example



Group walkthrough example



Edge Cases and Limitations

1. Negative Edge Weights

- Dijkstra assumes all edge weights are non-negative
- Negative weights can produce incorrect shortest paths
- Use Bellman–Ford instead

2. Disconnected Graphs

- Unreachable nodes remain at *distance* = ∞

Edge Cases and Limitations

3. Duplicate Priority Queue Entries

- A node may be inserted multiple times with different distances
- Outdated entries are skipped during extraction

4. When to Use Another Algorithm

- Equal weights → use BFS
- Weights restricted to 0 or 1 → use 0-1 BFS

0-1 BFS - An Exploration



Why Would We Ever Use 0-1 BFS?

Observation

Dijkstra works for all non-negative weights, but requires a priority queue and incurs a $\log V$ factor.

Special Case

If edge weights are restricted to $\{0, 1\}$:

- We do not need full priority ordering
- Only two distance transitions are possible

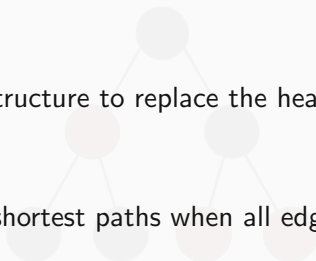
Why Would We Ever Use 0–1 BFS?

Key Idea

Exploit the weight structure to replace the heap with a deque.

Guarantee

0–1 BFS computes shortest paths when all edge weights are exactly 0 or 1.

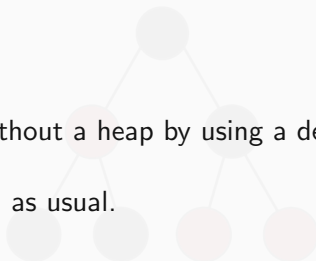


Algorithmic Theory

Core Idea

Simulate Dijkstra without a heap by using a deque.

Maintain $distance[v]$ as usual.

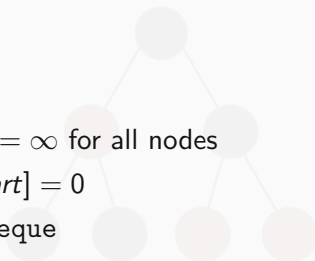


Algorithmic Theory

Algorithm

1. Initialize

- Set $distance[v] = \infty$ for all nodes
- Set $distance[start] = 0$
- Create empty deque
- Push $start$ to front



Algorithmic Theory

2. While deque not empty

- Pop node u from front
- For each neighbour v with weight $w \in \{0, 1\}$:
 - If $distance[u] + w < distance[v]$:
 - Update $distance[v]$
 - If $w = 0$: push v to front
 - If $w = 1$: push v to back

Algorithmic Theory

3. Stop when deque empty



Algorithmic Theory

Why It Works

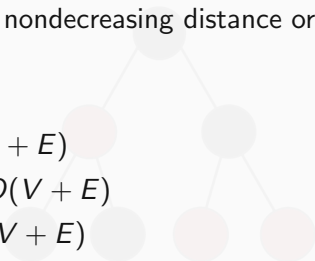
The deque preserves nondecreasing distance order.

Time Complexity

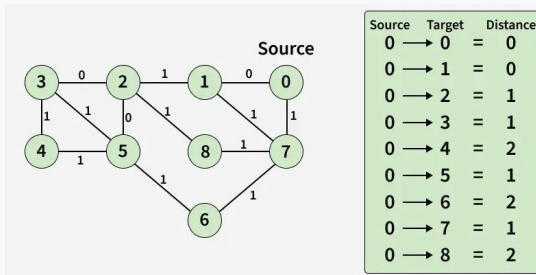
- Best case: $O(V + E)$
- Average case: $O(V + E)$
- Worst case: $O(V + E)$

Space Complexity

- $O(V + E)$



Group walkthrough example



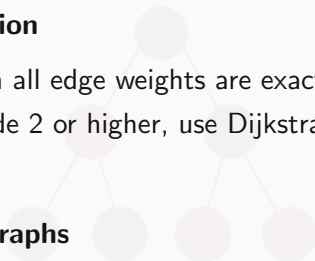
Edge Cases and Limitations

1. Weight Restriction

- Only valid when all edge weights are exactly 0 or 1
- If weights include 2 or higher, use Dijkstra (or Dial's algorithm)

2. Disconnected Graphs

- Unreachable nodes remain at *distance* = ∞



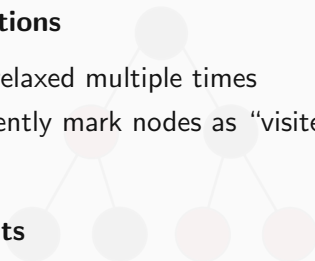
Edge Cases and Limitations

3. Multiple Relaxations

- Nodes may be relaxed multiple times
- Do not permanently mark nodes as “visited” after first processing

4. Negative Weights

- Not valid if negative edges are present

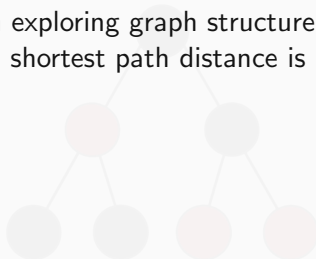


Choosing the Right Algorithm (Quick Rules)



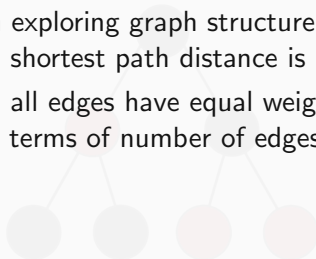
Choosing the Right Algorithm

→ **DFS:** Use when exploring graph structure such as connectivity or cycles, where shortest path distance is not required.



Choosing the Right Algorithm

- **DFS:** Use when exploring graph structure such as connectivity or cycles, where shortest path distance is not required.
- **BFS:** Use when all edges have equal weight and you need the shortest path in terms of number of edges.



Choosing the Right Algorithm

- **DFS:** Use when exploring graph structure such as connectivity or cycles, where shortest path distance is not required.
- **BFS:** Use when all edges have equal weight and you need the shortest path in terms of number of edges.
- **0-1 BFS:** Use when edge weights are restricted to exactly 0 or 1 and you need shortest path distances efficiently.

Choosing the Right Algorithm

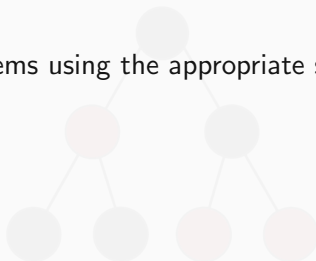
- **DFS:** Use when exploring graph structure such as connectivity or cycles, where shortest path distance is not required.
- **BFS:** Use when all edges have equal weight and you need the shortest path in terms of number of edges.
- **0-1 BFS:** Use when edge weights are restricted to exactly 0 or 1 and you need shortest path distances efficiently.
- **Dijkstra:** Use when edge weights are non-negative and may vary, and you need the shortest path from a source node.



Problem Sheet

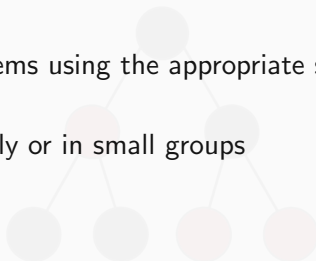
Problem Sheet

→ Solve the problems using the appropriate shortest-path algorithm

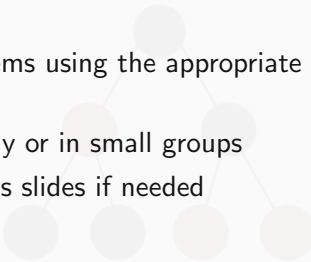


Problem Sheet

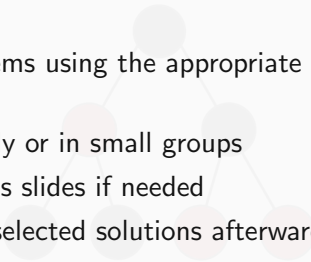
- Solve the problems using the appropriate shortest-path algorithm
- Work individually or in small groups

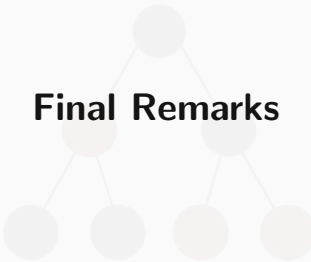


Problem Sheet

- 
- Solve the problems using the appropriate shortest-path algorithm
 - Work individually or in small groups
 - Refer to previous slides if needed

Problem Sheet

- 
- Solve the problems using the appropriate shortest-path algorithm
 - Work individually or in small groups
 - Refer to previous slides if needed
 - We will review selected solutions afterwards



Final Remarks

Further Reading

- 
- A document containing relevant supplementary + research reading will be available on the KCPC website (for this workshop) and the KCPC Discord Resource section for you to check out :D